

Explication du code

Bmp280.py

La classe et l'adresse I2C

Création de la classe BMP280, nous utilisons l'adresse I2C 0x76 pour ce capteur. Nous vérifions si le capteur est bien connecté.

```
class BMP280:
    I2C_ADDRESS = 0x76
    def __init__(self, i2c, address=0x76):
        self.i2c = i2c
        self.address = address
        devices = self.i2c.scan()
        if self.address not in devices:
            print(f"BMP280 non trouvé! Appareils détectés: {devices}")
        else: print("BMP280 trouvé!")
```

Initialisation hardware

Lecture des données d'étalonnage du capteur, début de la mesure, réglage de la vitesse d'échantillonnage.

```
self._lire_calibration()
self.i2c.writeto_mem(self.address, 0xF4, bytes([0x27]))
self.i2c.writeto_mem(self.address, 0xF5, bytes([0xA0]))
```

Lecture de calibration

Lecture des 24 octets de mémoire du capteur sur base de l'adresse 0x88 afin de récupérer les valeurs de correction propres au capteur.

```
def _lire_calibration(self):
    cal = self.i2c.readfrom_mem(self.address, 0x88, 24)
    self.dig_T1 = (cal[1] << 8) | cal[0]
    self.dig_T2 = self._signe((cal[3] << 8) | cal[2])
    self.dig_T3 = self._signe((cal[5] << 8) | cal[4])
    self.dig_P1 = (cal[7] << 8) | cal[6]
    self.dig_P2 = self._signe((cal[9] << 8) | cal[8])
    self.dig_P3 = self._signe((cal[11] << 8) | cal[10])
    self.dig_P4 = self._signe((cal[13] << 8) | cal[12])
    self.dig_P5 = self._signe((cal[15] << 8) | cal[14])
    self.dig_P6 = self._signe((cal[17] << 8) | cal[16])
    self.dig_P7 = self._signe((cal[19] << 8) | cal[18])
    self.dig_P8 = self._signe((cal[21] << 8) | cal[20])
    self.dig_P9 = self._signe((cal[23] << 8) | cal[22])
```

Fonction signe

Rectification des valeurs de calibration.

```
def _signe(self, valeur):  
    if valeur > 32767:  
        valeur -= 65536  
    return valeur
```

Lecture des données brutes

Lecture des données brutes du capteur dans les 6 octets depuis 0xF7, les 3 premiers sont la pression, les 3 suivants la température.

```
def _lire_brut(self):  
    data = self.i2c.readfrom_mem(self.address, 0xF7, 6)  
    pression_brute = ((data[0] << 16) | (data[1] << 8) | data[2]) >> 4  
    temp_brute = ((data[3] << 16) | (data[4] << 8) | data[5]) >> 4  
    return pression_brute, temp_brute
```

Calcul de la température

Application de la formule officielle pour passer de la valeur brute au °C.

```
def lire_temperature(self):  
    _, temp_brute = self._lire_brut()  
    var1 = (temp_brute / 16384.0 - self.dig_T1 / 1024.0) * self.dig_T2  
    var2 = (temp_brute / 131072.0 - self.dig_T1 / 8192.0) ** 2 * self.dig_T3  
    self.t_fine = var1 + var2  
    temperature = self.t_fine / 5120.0  
    return round(temperature, 2)
```

Calcul de la pression

Application de la formule officielle pour passer de la valeur brute à l'hectopascal (hPa), en prenant en compte la température.

```
def lire_pression(self):  
    pression_brute, _ = self._lire_brut()  
    self.lire_temperature()  
    var1 = self.t_fine / 2.0 - 64000.0  
    var2 = var1 * var1 * self.dig_P6 / 32768.0  
    var2 = var2 + var1 * self.dig_P5 * 2.0  
    var2 = var2 / 4.0 + self.dig_P4 * 65536.0  
    var1 = (self.dig_P3 * var1 * var1 / 524288.0 + self.dig_P2 * var1) / 524288.0  
    var1 = (1.0 + var1 / 32768.0) * self.dig_P1  
    if var1 == 0:  
        return 0  
    pression = 1048576.0 - pression_brute  
    pression = ((pression - var2 / 4096.0) * 6250.0) / var1  
    var1 = self.dig_P9 * pression * pression / 2147483648.0  
    var2 = pression * self.dig_P8 / 32768.0  
    pression = pression + (var1 + var2 + self.dig_P7) / 16.0  
    return round(pression / 100.0, 2)
```

capteurs.py

Initialisation du sonar

Initialisation des pin "trig" et "echo" pour le déclenchement et la réception du signal, mise à zéro de "trig".

```
class HCSR04:
    def __init__(self, trig_pin, echo_pin):
        self.trig = machine.Pin(trig_pin, machine.Pin.OUT)
        self.echo = machine.Pin(echo_pin, machine.Pin.IN)
        self.trig.value(0)
        time.sleep_ms(100)
```

Mesure de distance

Calcule la distance entre le capteur et un obstacle, selon le temps que mettent les ultrasons à revenir. Si le signal ne revient pas, c'est qu'il n'y a pas d'obstacle.

```
def mesurer_distance(self):
    self.trig.value(0)
    time.sleep_us(2)
    self.trig.value(1)
    time.sleep_us(10)
    self.trig.value(0)
    timeout = time.ticks_us()
    while self.echo.value() == 0:
        if time.ticks_diff(time.ticks_us(), timeout) > 30000:
            return -1
    debut = time.ticks_us()
    while self.echo.value() == 1:
        if time.ticks_diff(time.ticks_us(), debut) > 30000:
            return -1
    fin = time.ticks_us()
    duree = time.ticks_diff(fin, debut)
    distance = duree / 58.0
    return round(distance, 1)
```

Mesure moyenne

Fait la moyenne de 3 mesures pour une plus grande fiabilité.

```
def mesurer_moyenne(self, nb_mesures=3):
    mesures = []
    for _ in range(nb_mesures):
        dist = self.mesurer_distance()
        if dist != -1 and dist <= 400:
            mesures.append(dist)
        time.sleep_ms(20)
    if mesures:
        return round(sum(mesures) / len(mesures), 1)
    else:
        return -1
```

main.py

Classe Moteurs

Initialisation des moteurs et de leur fréquence, définition du PWM qui permet la modulation de vitesse selon l'envoi de courant à une certaine fréquence.

```
class Moteurs:
    VITESSE_MAX = 65535
    def __init__(self, in1, in2, in3, in4):
        self.in1 = machine.PWM(machine.Pin(in1))
        self.in2 = machine.PWM(machine.Pin(in2))
        self.in3 = machine.PWM(machine.Pin(in3))
        self.in4 = machine.PWM(machine.Pin(in4))
        self.in1.freq(1000)
        self.in2.freq(1000)
        self.in3.freq(1000)
        self.in4.freq(1000)
```

Conversion vitesse

Conversion de la vitesse du pourcentage (0 - 100) vers la grandeur PWM (0 - 65535) avec protection empêchant le dépassement des valeurs.

```
def _vitesse_pwm(self, pourcentage):
    pourcentage = max(0, min(100, pourcentage))
    return int(pourcentage / 100.0 * self.VITESSE_MAX)
```

Commandes des moteurs

Fait aller les moteurs en avant ou en arrière en fonction de la direction souhaitée.

```
def avancer(self, vitesse=70):
    pwm = self._vitesse_pwm(vitesse)
    self.in1.duty_u16(pwm)
    self.in2.duty_u16(0)
    self.in3.duty_u16(pwm)
    self.in4.duty_u16(0)
    def tourner_gauche(self, vitesse=60):
        pwm = self._vitesse_pwm(vitesse)
        self.in1.duty_u16(0)
        self.in2.duty_u16(pwm)
        self.in3.duty_u16(pwm)
        self.in4.duty_u16(0)
```

Configuration des pins

Définition des GPIO connectés pour meilleure lisibilité du code.

```
I2C_SDA = 0
I2C_SCL = 1
PIN_IN1 = 13
PIN_IN2 = 12
PIN_IN3 = 11
PIN_IN4 = 10
PIN_TRIG = 9
PIN_ECHO = 8
PIN_LED_VERTE = 14
PIN_LED_ROUGE = 15
```

Seuils et paramètres

Définition des Variable, seuils et paramètre.

```
DISTANCE_DANGER      = 15
DISTANCE_ATTENTION   = 30

VITESSE_NORMALE      = 70
VITESSE_LENTE        = 40
VITESSE_ROTATION     = 55

TEMP_MIN_OK          = 18.0
TEMP_MAX_OK          = 28.0
PRESSION_MIN_OK       = 980.0
PRESSION_MAX_OK       = 1030.0

INTERVALLE_BMP       = 3.0
```

Initialisation des composants

Création de bus I2C a 400kHz et initialisation des composant avec les différente class.

```
i2c = machine.I2C(0, sda=machine.Pin(I2C_SDA), scl=machine.Pin(I2C_SCL), freq=400000)
bmp = BMP280(i2c, address=119)
moteurs = Moteurs(PIN_IN1, PIN_IN2, PIN_IN3, PIN_IN4)
sonar = HCSR04(PIN_TRIG, PIN_ECHO)
led_verte = machine.Pin(PIN_LED_VERTE, machine.Pin.OUT)
led_rouge = machine.Pin(PIN_LED_ROUGE, machine.Pin.OUT)
```

Séquence de démarrage

Fait clignoter les LEDs 3 fois.

```
for _ in range(3):
    led_verte.value(1)
    led_rouge.value(1)
    time.sleep_ms(300)
    led_verte.value(0)
    led_rouge.value(0)
    time.sleep_ms(300)
```

Fonction évaluation environnement

Evalue l'environnement en fonction de la température et pression détecter par le capteur et des Seuils définie.

```
def evaluer_environnement(temperature, pression):
    temp_ok      = TEMP_MIN_OK <= temperature <= TEMP_MAX_OK
    pression_ok  = PRESSION_MIN_OK <= pression <= PRESSION_MAX_OK
    return temp_ok and pression_ok
```

Fonction LEDs

Actualisation des LEDs en fonction de l'évaluation de l'environnement.

```
def mettre_a_jour_leds(environnement_ok):
    if environnement_ok:
        led_verte.value(1)
        led_rouge.value(0)
    else:
        led_verte.value(0)
        led_rouge.value(1)
```

Fonction évitement d'obstacles

Modification de la vitesse en fonction de la distance établie par le capteur et change de direction si l'obstacle est trop proche.

```
def eviter_obstacle(distance):
    if distance == -1:
        return "libre"
    elif distance <= DISTANCE_DANGER:
        moteurs.arreter()
        time.sleep_ms(200)
        moteurs.reculer(VITESSE_LENTE)
        time.sleep_ms(500)
        moteurs.arreter()
        time.sleep_ms(100)
        if random.randint(0, 1) == 0:
            moteurs.tourner_gauche(VITESSE_ROTATION)
            direction = "gauche"
        else:
            moteurs.tourner_droite(VITESSE_ROTATION)
            direction = "droite"
        time.sleep_ms(600)
        moteurs.arreter()
        return f"danger! recul + rotation {direction}"
    elif distance <= DISTANCE_ATTENTION:
        moteurs.avancer(VITESSE_LENTE)
        return f"attention ({distance}cm) - ralenti"
    else:
        return "libre"
```

Boucle principale

Applique en boucle les différentes fonctions pour avoir un robot complètement autonome.

```
dernier_bmp = time.time()
environnement_ok = True
moteurs.avancer(VITESSE_NORMALE)

while True:
    distance = sonar.mesurer_distance()
    action = eviter_obstacle(distance)
    if action == "libre":
        moteurs.avancer(VITESSE_NORMALE)
    maintenant = time.time()
    if maintenant - dernier_bmp >= INTERVALLE_BMP:
        try:
            temperature, pression = bmp.lire_tout()
            environnement_ok = evaluer_environnement(temperature, pression)
            mettre_a_jour_leds(environnement_ok)
            statut = "OK" if environnement_ok else "ANORMAL"
            txt_distance = f"{distance}cm" if distance != -1 else "Erreur
capteur"

            print(
                f"Temp: {temperature}°C | "
                f"Pression: {pression}hPa | "
                f"Environnement: {statut} | "
                f"Distance: {txt_distance}"
            )
        except Exception as e:
            print(f"Erreur BMP280: {e}")
        dernier_bmp = maintenant
    if distance != -1 and action != "libre":
        print(f"[ALERTE] Distance active: {distance}cm | Action: {action}")
    time.sleep_ms(50)
```

Algorithme

